CE-355 Lab 5
Real-Time Camera Image Processor

1. Introduction

Image filters are everywhere: the grayscale and sepia effects in a photo app, the edge detection a self-driving car uses to find lane lines, the cartoon effects in a video call. Each one works the same basic way. It takes every pixel of an image, applies a small calculation to it, and produces a new pixel. In this lab, you will build an FPGA design that does this to live video. A camera attached to the DE25-Standard captures 640 × 480 video, the FPGA applies one of eight filters to every pixel, and the result appears on an HDMI monitor in real time. Two push buttons step through the filters, and the seven-segment display shows which filter is active.

The camera and the monitor run on separate, unrelated clocks. The camera sends pixels whenever it is ready, and the monitor needs pixels at its own steady rate. To connect them, the design stores each incoming frame in a **frame buffer**, a block of on-chip memory large enough to hold a whole picture. However, the FPGA has only a limited amount of on-chip memory, so the image data must be **compressed** to fit: each pixel is reduced from 24 bits to 20 bits before it is stored. The camera side compresses each pixel and writes it into the frame buffer. The display side reads each pixel back out, expands it back to 24 bits, applies the selected filter, and sends the result to the HDMI output.

This lab introduces several new VHDL and RTL concepts:

- **Inferred block RAM:** describing a dual-port memory in VHDL so that Quartus builds it from the FPGA's dedicated memory blocks, and using it as a frame buffer.
- **Records and packages:** using a record type to group a pixel's red, green and blue values, and a shared package of conversion functions that every filter uses.
- **Nested generate statements:** using **if-generate** inside a **for-generate** to build a different filter at each position in a bank.
- **Valid handshakes:** passing a valid signal alongside the data, so every stage of a pipeline knows exactly when a new pixel has arrived.
- **Fixed-point coefficients and saturation:** multiplying by fractional constants with integer hardware, and clamping results that would otherwise overflow.
- **Debouncing:** turning a noisy mechanical button press into exactly one clean step.
- **Reading binary files in a testbench:** loading a real BMP image into the simulation and streaming it into the design as if it came from the camera.

It also introduces two new parts of an FPGA system:

- **A camera input:** the Terasic **D8M** camera daughter card, its configuration, and its own pixel clock.

- **Multiple clock domains:** a PLL that generates both the system clock and the camera's reference clock, and a safe way to bring the camera's data, which arrives on a different clock, into the system clock domain.

The camera interface, the HDMI output, the frame-buffer write logic and the top-level system with its PLL are provided for you. You will complete the frame buffer memory, the filter bank, the seven filters, the seven-segment display, the image processor wiring and the testbench. You will then simulate the design on real photographs, and synthesize, program and test it on the DE25-Standard with the camera.

By completing this lab, you will:

- Describe a dual-port block RAM in VHDL and use it as a frame buffer.
- Use records, packages and functions to keep pixel arithmetic readable.
- Implement image filters, including fixed-point coefficients, saturation, and a filter that remembers the previous pixel.
- Use nested generate statements to build a bank of different filters behind one multiplexer.
- Use a valid signal to keep a pipeline aligned with the data passing through it.
- Debounce a push button with a counter.
- Explain why a design needs a frame buffer and a PLL when the camera and display run on different clocks.
- Read and write binary image files in a testbench.
- Simulate, program and test a real-time FPGA video processing system.

2. Background: Digital Images and Image Filters

2.1 Pixels and Color

A digital image is a grid of **pixels**, small squares that each have a single color. The images in this lab are **640 × 480**: 640 pixels across and 480 rows down, or 307,200 pixels in total.

Each pixel's color is described by how much **red**, **green** and **blue** light it contains. Each of the three amounts is called a **channel**, and each is stored as an 8-bit number from 0 (none) to 255 (full brightness). This format is called **RGB888**, for the 8 bits in each channel, and it uses 24 bits per pixel. (Note that this would require a total of 7.37Mbits of memory to store, which is over capacity for the FPGA memory.)

When all three channels are equal, the pixel is a shade of gray. The higher the values, the brighter the pixel. A few examples:

Color	Red	Green	Blue
Black	0	0	0
White	255	255	255

Color	Red	Green	Blue
Pure red	255	0	0
Yellow (red + green)	255	255	0
Mid gray	128	128	128

2.2 Raster Order

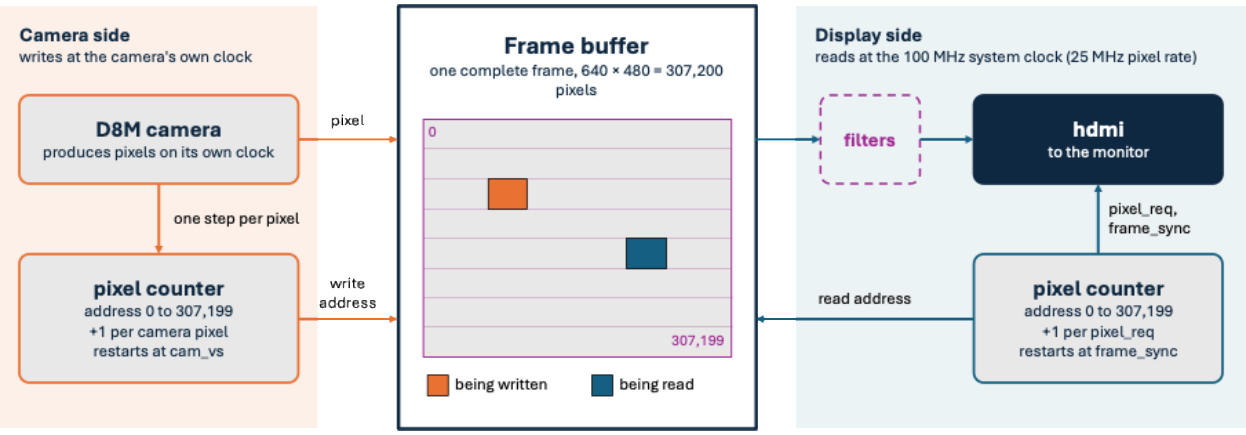
Cameras and monitors both handle pixels one at a time in the same order, called **raster order**: the top row from left to right, then the next row, and so on down to the bottom-right pixel. A complete image sent this way is one **frame**. Video is a stream of frames, here 60 per second.

Because pixels always arrive in raster order, a pixel's position in the frame is simply how many pixels have come before it in that frame. Pixel number *n* is in row *n* ÷ 640 and column *n* mod 640. This is how the design keeps track of position: it counts pixels, starting from 0 at the beginning of each frame.

2.3 The Frame Buffer

In Lab 4, every pixel was computed at the exact moment the monitor needed it. That cannot work with a camera. The camera produces pixels on its own clock, at its own timing, and has no idea when the monitor wants them. If the design tried to pass each camera pixel straight to the monitor, the two would drift apart and the picture would tear.

The solution is to store each image frame into a memory **frame buffer**. The camera **writes** each pixel into the buffer as it arrives, at the address equal to its position in the frame. Separately, the display side **reads** pixels out of the buffer in raster order, at the rate the monitor needs. The two sides never have to agree on timing, because the buffer sits between them.



A full frame at 24 bits per pixel would need 640 × 480 × 24 ≈ 7.4 million bits, more on-chip memory than the FPGA has. So the frame buffer stores each pixel in **20 bits** instead, using a format called **RGB686**: 6 bits of red, 8 bits of green, and 6 bits of blue. Green keeps all 8 bits because the human eye is most sensitive to green, so that is where the detail matters most. Red and blue lose their 2 lowest bits, a change too small to notice in most images.

2.4 Image Filters

An **image filter** turns every pixel of an image into a new pixel. In this lab you will build 7 image filters (8 in total including the passthrough). Most of them are **point operations**: each output pixel depends only on the input pixel at the same position. Two of them also look at the previous pixel in the row, which lets them detect edges.

- **Passthrough.** This is provided to you as a template, where the output is the input, unchanged. This is filter 0, and it shows the camera image as it is.
- **Grayscale.** A gray pixel has equal red, green and blue values, so to remove the color, replace all three channels with the pixel's **brightness**. This lab uses the simple average of the three channels:

$$\text{gray} = \frac{R + G + B}{3}$$

- **Sepia.** Sepia gives a photo the warm brown tone of an old print. Each output channel is a weighted mix of all three input channels, using a standard set of coefficients:

$$R' = 0.393\,R + 0.769\,G + 0.189\,B$$
$$G' = 0.349\,R + 0.686\,G + 0.168\,B$$
$$B' = 0.272\,R + 0.534\,G + 0.131\,B$$

  The red row's coefficients add up to more than 1, so a bright pixel can produce a result above 255. Those results must be **saturated** to 255.

- **Color swap.** Exchange the red and blue channels. A red object turns blue, and a blue sky turns orange.
- **Invert.** Replace each channel with its opposite, producing a photographic negative:

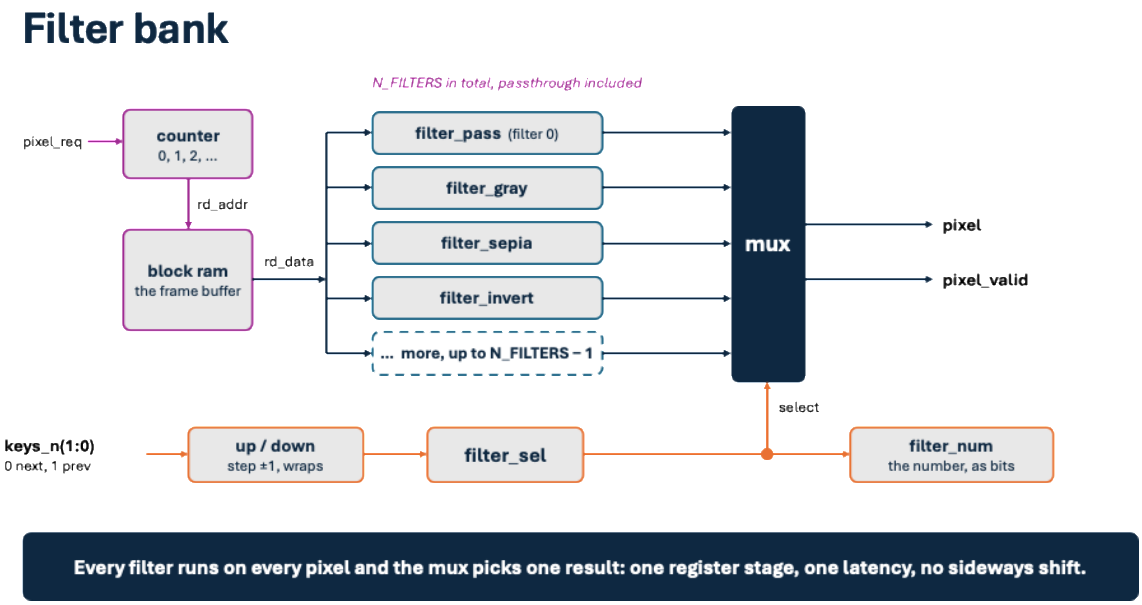
$$R' = 255 - R, \qquad G' = 255 - G, \qquad B' = 255 - B$$

- **Threshold.** Turn the image into pure black and white. Compute the brightness as in the grayscale filter, then output **white** if it is above a fixed level and **black** otherwise.
- **Edge detection.** An **edge** is a place where the image changes sharply, like the outline of an object against a background. Moving along a row, the brightness barely changes on a flat surface, but it jumps at an edge. So the filter compares each pixel's brightness with the brightness of the pixel just before it:

$$\text{change} = |\,\text{gray(this pixel)} - \text{gray(previous pixel)}\,|$$

  If the change is larger than a set level, the pixel is on an edge and is drawn **black**. Otherwise it is drawn **white**. The result looks like a pen sketch of the scene. Because it only compares neighbors along a row, this filter finds edges that run up and down (vertical edges) but not edges that run left to right. For a camera pointed at a room, most edges are vertical anyway.

- **Cartoonify.** A cartoon has two features: large areas of flat color, and dark outlines. This filter produces both:
    1. **Flat color (posterize).** Reduce each channel from 256 possible values to just 4 evenly spaced levels (0, 85, 170 and 255), rounding each value to the **nearest** level. Smooth shading becomes a few flat bands of color.
    2. **Outlines.** Use the same edge test as the edge filter. Where there is an edge, draw a black pixel instead of the flat color.



3. Project Setup

You will use **ModelSim** to simulate and verify your design, and **Altera Quartus Prime** to compile, synthesize and program it on the DE25-Standard FPGA development board.

The lab uses the same directory structure as the earlier labs:

```
Lab 5/
│
├── vhdl/  — Contains the VHDL source files for the design.
│   └── d8m/  — Terasic Verilog files for the camera configuration.
├── sim/   — Contains the ModelSim command file, test images and simulation output.
└── syn/   — Contains the Quartus Prime project, constraints and PLL IP.
```

The design files are:

File	Status	Description
pixel_pkg.vhd	Provided	The pixel_t record type and the pixel format conversion functions

File	Status	Description
bram.vhd	**TODO**	Dual-port block RAM, used as the frame buffer
video_store.vhd	Provided	Writes camera pixels into the frame buffer
filters.vhd	**TODO**	Reads the frame buffer, runs every filter, and selects one with the buttons
filter_pass.vhd	Provided	Passthrough filter, and the template for every other filter
filter_gray.vhd, filter_sepia.vhd, filter_swap.vhd, filter_invert.vhd, filter_threshold.vhd, filter_edge.vhd, filter_cartoonify.vhd	**TODO (new files)**	The 7 filters, which you create
digit_display.vhd	**TODO**	One seven-segment digit, from Lab 1, with a register added
image_processor.vhd	**TODO**	Connects the video store, frame buffer, filters and displays
d8m.vhd and d8m/*.v	Provided	D8M camera interface and configuration
hdmi.vhd	Provided	640 × 480 video timing, HDMI transmitter pins and I2C setup, from Lab 4
image_processor_system.vhd	Provided	Board-level top: PLL, camera, image processor and HDMI output
image_processor_tb.vhd	**TODO**	Testbench that streams a BMP image through the design and writes the results
sim/u2_0.bmp, sim/u2_1.bmp, sim/u2_2.bmp	Provided	640 × 480 test images
syn/VIDEO_PLL.ip	Provided	Quartus PLL IP: 50 MHz in, 20 MHz and 100 MHz out

*NOTE: Throughout this assignment, please refer to the **DE25-Standard Development Kit User Manual**, the **DE25-Standard Getting Started Guide**, the **D8M camera user manual**, and the **ADV7513 HDMI transmitter datasheet**, which contain more details on the board setup, pinouts, camera and HDMI transmitter.*

4. System Design

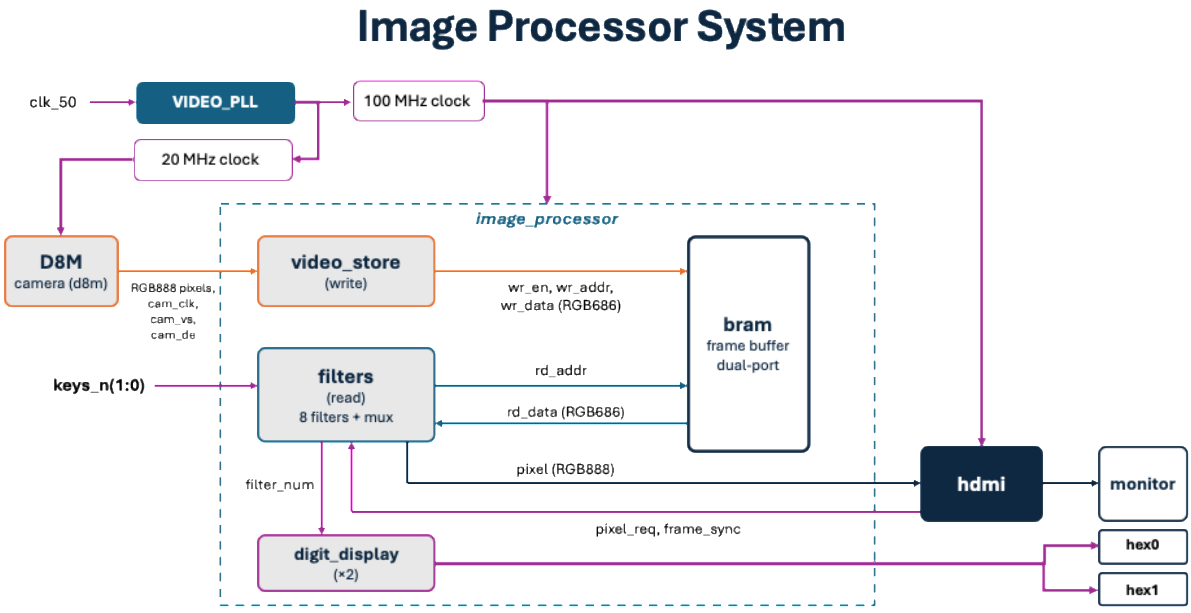
4.1 Architecture

The following is the system architecture for the image processor design:

- **d8m** (provided): configures the camera, receives its raw sensor data, and converts it into an ordinary stream of RGB888 pixels in raster order, with a frame-valid signal (cam_vs), a pixel-valid signal (cam_de) and the camera's pixel clock (cam_clk).
- **video_store** (provided): the **write side**. It counts camera pixels to get each pixel's frame-buffer address, converts each pixel from RGB888 to RGB686, and writes it into the frame buffer.

- **bram:** the **frame buffer**, a dual-port memory. One port writes camera pixels, and the other port reads pixels for the display, at the same time.
- **filters:** the **read side**. It keeps its own pixel counter to read the frame buffer in raster order, one pixel each time the display asks, and passes every pixel through all eight filters at once. A multiplexer then picks the output of the filter currently selected with the buttons.
- **digit_display** (two copies): shows the selected filter number on HEX1 and HEX0.
- **hdmi** (provided, from Lab 4): generates the 640 × 480, 60 Hz video timing, asks for each pixel with pixel_req, signals the gap between frames with frame_sync, and drives the HDMI transmitter.

The write side and the read side never communicate directly. The frame buffer is the only connection between them.



4.2 The D8M Camera

The **D8M** is a Terasic camera daughter card that plugs into the DE25-Standard's GPIO expansion header. It contains two chips:

- **An image sensor**, the camera itself. It captures images as a grid of light-sensitive cells and sends them out over **MIPI CSI-2**, the high-speed serial interface used by phone cameras.
- **A MIPI bridge** (Toshiba TC358748), which converts the MIPI serial stream into an ordinary parallel bus the FPGA can read: 10 data bits per pixel, line and frame sync signals, and a pixel clock.

Both chips must be **configured over I2C** before they produce an image: the sensor's resolution, exposure and orientation, and the bridge's data format and timing. Each has its own I2C bus.

The provided Terasic Verilog files in vhdl/d8m/ contain the register settings and send them at startup, which is configured to generate 640 x 480 image frames.

The sensor does not capture full color at every pixel. Each light-sensitive cell sits behind a red, green or blue filter, arranged in a repeating 2 × 2 pattern called a **Bayer pattern**, so each raw pixel measures only one color. The provided d8m.vhd performs **demosaicing**: it estimates the two missing colors at each pixel from its neighbors, producing a full RGB888 pixel. The resulting stream is 640 × 480 at a **25 MHz** pixel clock that comes from the bridge.

4.3 The Frame Buffer Memory

The FPGA's on-chip memory is built from **M20K blocks**, each holding 20 kbits, organized as up to 1,024 words of 20 bits. The device has 358 of them.

A 640 × 480 frame is 307,200 pixels, which needs 307,200 ÷ 1,024 = **300 blocks** at any word width from 1 to 20 bits. The number of blocks is set by the number of pixels, not by how many bits each one uses, up to 20 bits. This is why the frame buffer uses the 20-bit RGB686 format: it uses every bit of the 300 blocks. A 24-bit pixel would need a second block for every 1,024 pixels, 600 blocks in total, which does not fit.

The memory is **dual-port**: it has a separate write port (address, data and write enable) and read port (address and data), which work at the same time. The read has a **latency of one clock cycle**: the read address is captured on a clock edge, and the data for that address appears on the output during the next cycle.

4.4 Clocking: The PLL and Two Clock Domains

The design uses 3 clocks:

Clock	Frequency	Source	Used by
Board oscillator	50 MHz	DE25-Standard	Input to the PLL, and the camera's startup logic
System clock	100 MHz	PLL output 1	The image processor and HDMI output
Camera reference clock	20 MHz	PLL output 0	Sent to the D8M's MIPI bridge
Camera pixel clock	25 MHz	D8M MIPI bridge	The camera's pixel stream

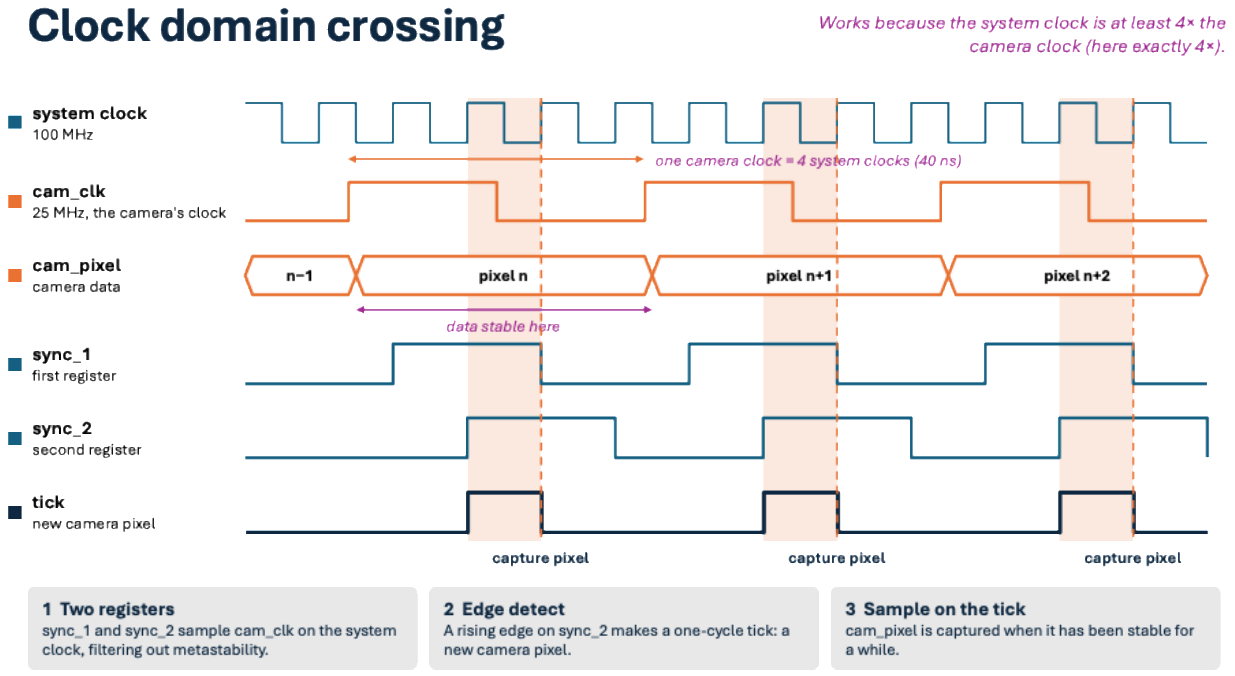
The provided VIDEO_PLL IP takes the 50 MHz board oscillator and produces two clocks:

- **100 MHz system clock** powers all the system modules. The HDMI module divides it by 4 to get the 25 MHz pixel rate that the 640 × 480, 60 Hz video mode requires, and generates the HDMI transmitter clock from the same divider.
- **20 MHz camera clock** is used by the D8M's MIPI bridge as an external reference clock to run its own internal PLL. The FPGA sends it out on the mipi_refclk pin.

Moving data from one clock domain to another needs care. If a register samples a signal at the exact moment it changes, the register can briefly settle to an invalid value, called **metastability**.

The provided video_store handles this with **oversampling**. It treats the camera clock as an ordinary signal, samples it on the 100 MHz system clock through two registers to filter out metastability, and detects each rising edge to make a one-cycle “new camera pixel” tick. It then samples the camera’s pixel data on that tick, when the data has been stable for a while. This only works if the system clock is at least **4 times** faster than the camera clock, so each camera clock cycle is sampled several times. At 100 MHz and 25 MHz, the ratio is exactly 4.

This is the design’s only clock domain crossing. Everything after video_store runs on the system clock.



4.5 Handshaking Between Blocks

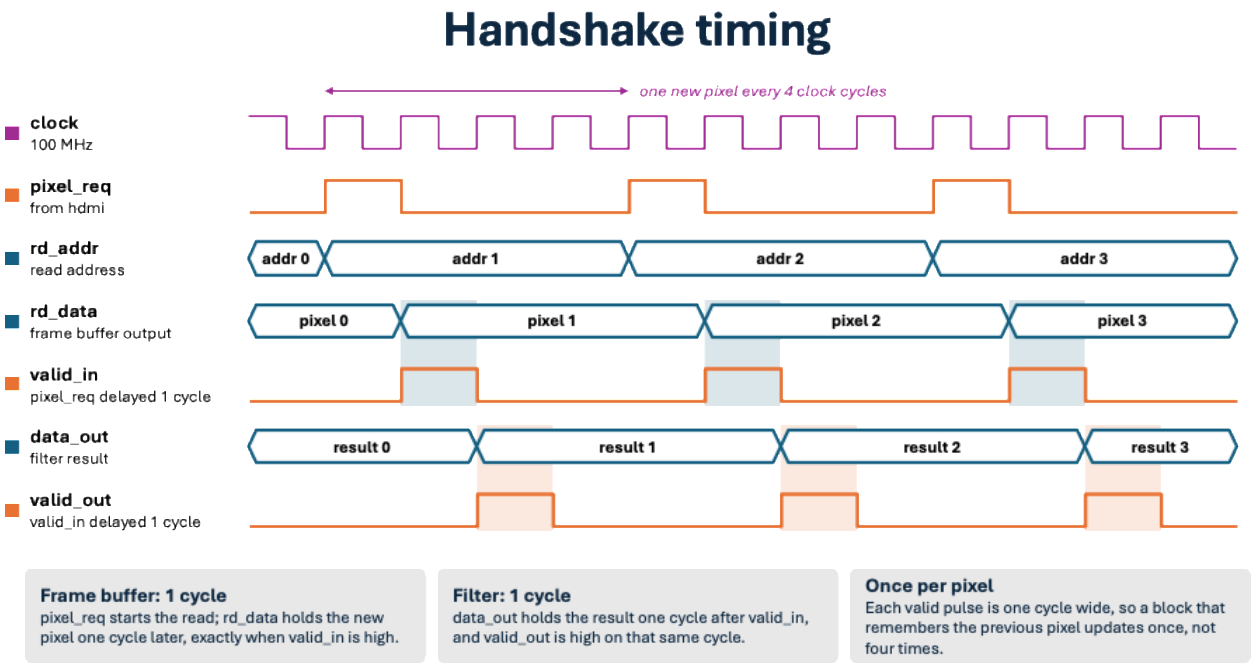
When one block passes data to another, the receiving block needs to know when the new data is ready. In this design that is not obvious. The blocks all run on the 100 MHz system clock, but a new pixel arrives only once every 4 clock cycles, because that is the display’s pixel rate. On the other 3 cycles, the data on the wires is still the previous pixel. Looking only at the data, a block cannot tell a new pixel from the same pixel seen again.

The solution is a **handshake**: a signal that travels alongside the data and says “this is a new value, use it now”. In this design it is called **valid**:

- The sending block sets **valid_out** high for exactly **one clock cycle**, the cycle when its data output holds a new pixel.
- The receiving block sees this as its **valid_in**, and only processes data_in on the cycles when valid_in = '1'. On every other cycle, it ignores data_in.
- When the receiving block has finished, it passes the handshake on: it sets its own valid_out high for one cycle, on the cycle its result is ready.

Every filter has the same pair of handshake ports, valid_in and valid_out, next to its data ports, data_in and data_out. This means every filter fits into the same place in the design, and the blocks connect in a chain, with each block's valid_out driving the next block's valid_in:

The handshake also means that the receiving block does not need to know how long the sending block takes. For example, the testbench's output process does not count clock cycles after it asks for a pixel. It waits until pixel_valid = '1' and then takes the pixel. If a filter were given an extra register stage, the testbench would not need to change, as long as the filter delayed its valid_out by the same amount.



5. Design Implementation

In this section, you will complete the **TODO** sections and create the seven filter files. Work from the bottom of the hierarchy up: the memory, the seven-segment digit, the filters, the filter bank, and finally the image processor wiring.

5.1 Implement the Frame Buffer (bram)

Complete bram.vhd. Quartus recognizes certain styles of VHDL as a memory and builds them from M20K blocks. For a dual-port RAM:

5.2 Implement the Seven-Segment Digit (digit_display)

Complete digit_display.vhd. This is the seven-segment decoder from Lab 1, except we are introducing a clocked process to register the output. It takes a 4-bit value din and drives the seven segments of one digit, showing the hexadecimal digits 0 to F. The image processor uses two copies: HEX0 shows the low 4 bits of the filter number, and HEX1 shows the high 4 bits.

5.3 Create the Seven Filters

Each filter is a separate entity in its own file. Use **filter_pass.vhd** as the template**.** Every filter must have the same entity ports as the passthrough. The **pixel_pkg.vhd** package provides functions to pack and unpack the RGB pixels, including some constant definitions like PIXEL_BLACK and PIXEL_WHITE.

Port	Direction	Width	Meaning
clock	in	1	System clock
reset	in	1	Asynchronous reset, active high
data_in	in	20	Pixel from the frame buffer, RGB686
valid_in	in	1	High for the one cycle when data_in is a new pixel
data_out	out	24	Filtered pixel, RGB888
valid_out	out	1	High for the one cycle when data_out is a new pixel

Grayscale (filter_gray)

Compute gray = (R + G + B) ÷ 3, and set all three output channels to gray. Each channel is 8 bits (0 to 255), but their sum can reach 765, which needs **10 bits**. Treat each channel as unsigned, resize each one to 10 bits, and then add. Divide by 3 builds a real divider circuit, which is acceptable here, but notice the cost in the resource report.

Sepia (filter_sepia)

Sepia gives a photo the warm brown tone of an old print. Each output channel is a weighted mix of all three input channels, using a standard set of coefficients. The coefficients are fractions, and there is no floating point in hardware, so we use **fixed-point** coefficients by scaling each coefficient by 256 and round it to an integer**.** For example, 0.393 × 256 ≈ 100 and 0.769 × 256 ≈ 197. To avoid overflow, the sum of three needs 18 bits, so resize the products before adding. Then divide by 256 (shift-right 8 bits) to undo the scaling, then saturate to 255. The full set is:

$$R' = \frac{(100\,R + 197\,G + 48\,B)}{256} \quad G' = \frac{(89\,R + 176\,G + 43\,B)}{256} \quad B' = \frac{70\,R + 137\,G + 34\,B}{256}$$

Color Swap (filter_swap)

Build the output record with red and blue exchanged: the output's red is the input's blue, and the output's blue is the input's red. Green is unchanged. This filter is pure wiring: it uses no logic at all.

Invert (filter_invert)

For each channel, compute 255 − value, treating the channel as unsigned. The result always fits in 8 bits, since the channel is between 0 and 255.

Threshold (filter_threshold)

Compute the brightness exactly as in the grayscale filter. If it is greater than the threshold level, output PIXEL_WHITE, otherwise PIXEL_BLACK. Give the entity a generic named THRESHOLD with a default of **100**, so the level can be changed without editing the code.

Edge Detection (filter_edge)

This is the first filter that remembers something: the previous pixel's brightness. Add a signal to hold the previous pixel's brightness. Compute this pixel's brightness as in the grayscale filter, then compute the absolute difference between this pixel's brightness and the previous one. If the difference is greater than the edge level (default 24), output PIXEL_BLACK, otherwise PIXEL_WHITE.

Cartoonify (filter_cartoonify)

This filter combines the edge filter with posterizing. First, compute the brightness, the absolute difference and the edge test exactly as in the edge filter, including remembering the previous brightness. Where there is an edge, output PIXEL_BLACK. Where there is no edge, posterize each channel (RGB) independently. With L = 2^POSTER_BITS (2^2 = 4) levels, round each channel value to the nearest level, then convert that level back to the 0 to 255 range:

$$n = \frac{v \times (L-1) + 127}{255} \qquad \text{(which level is closest, rounded)}$$

$$v' = \frac{n \times 255}{L-1} \qquad \text{(back to the 0 to 255 range)}$$

5.4 Implement the Filter Bank (filters)

Complete filters.vhd TODO sections that include:

- Implement a clocked process to address the block RAM.
    - Initialize the address when frame_sync = '1', the display is between frames.
    - Each time pixel_req = '1', the display wants the next pixel, so increase the address by 1; when it reaches N_PIXELS − 1, wrap back to 0.
    - Drive valid_in when the data is ready, one cycle after the address is set.
- Implement a for-generate loop with if-generate to select more filters.
    - Add a **component declaration** for each of your seven filters, alongside the one for filter_pass.
    - Replace the single filter_pass instance with a **for-generate** loop over 0 to N_FILTERS − 1. Use an **if-generate** to build a different filter at each index *i.*

- Implement a clocked process to step through the filters when the up/down keys are pressed.
    - The two buttons step through the filters: keys_n(0) moves to the next filter, and keys_n(1) moves to the previous one. Both wrap around at the ends of the list.
    - Use a clock counter to ensure the button is held long enough (DEBOUNCE_TICKS) to avoid debounce effects.

5.5 Implement the Image Processor

Complete image_processor.vhd. This is a structural design connecting the video_store, bram, filters and digit_display.

6. Simulation

Before programming the FPGA, verify the design in ModelSim. The testbench simulates image_processor on its own, without the camera or HDMI modules. It reads a real 640 × 480 photograph from a BMP file, streams it into the camera port exactly as the D8M would, then reads the frame back out through each filter and writes each result to a new BMP file. You check the design by opening the output images.

6.1 Complete the Testbench

The provided **image_processor_tb.vhd** already contains the BMP reading and writing procedures, the signal declarations, and the output process, which plays the role of the display. Read through them before you begin. You will complete four TODO sections.

Step 1: Instantiate the DUT. Instantiate image_processor with the label **dut**, passing the testbench's IMG_W, IMG_H, ADDR_W, N_FILTERS and DEBOUNCE_TICKS constants, and connecting every port to the testbench signal of the same name.

Step 2: Generate the clocks. Write two clock processes:

- The **system clock**, clock, with period SYS_PERIOD (10 ns, 100 MHz).
- The **camera clock**, cam_clk, with period CAM_PERIOD (40 ns, 25 MHz). On the board this clock is unrelated to the system clock, so give it an arbitrary phase offset, for example by waiting a few nanoseconds before it starts. This tests that video_store works for any phase.

Step 3: Generate the reset. The DUT's reset is active high. Hold reset high for a few system clock cycles at the start of the simulation, then release it.

Step 4: Write the input process. This process plays the role of the camera. It streams the image into the design once for each filter, and coordinates with the output process through two handshake signals: frame_ready (input → output: the frame is in the buffer and the filter is selected) and frame_done (output → input: the frame has been captured and written to a file).

1. **Wait for reset** to finish before starting.
2. **Read the image** into the image_frame variable using the read_bmp procedure.
3. **For each filter**, from 0 to N_FILTERS − 1:
   - **Stream the frame,** one pixel per rising edge of cam_clk. Raise cam_vs to start the frame, then raise cam_de and drive cam_pixel with each of the frame's pixels in order, then finally lower cam_de and cam_vs at the end.

```
wait until rising_edge(cam_clk);
cam_vs <= '1';
wait until rising_edge(cam_clk);
cam_de <= '1';

for i in 0 to N_PIXELS - 1 loop
   cam_pixel <= image_frame(i);
   wait until rising_edge(cam_clk);
end loop;

cam_de <= '0';
cam_vs <= '0';
wait until rising_edge(cam_clk);
```

   - **Process the frame** by setting frame_ready to '1', then wait until frame_done is '1', set frame_ready back to '0', and wait until frame_done returns to '0'.
   - **Select the next filter** by seting keys_n to "10" (key 0 pressed), hold it for more than DEBOUNCE_TICKS cycles, then release it with "11". Note that the testbench uses DEBOUNCE_TICKS = 2^5 instead of the board's 2^{20}, so each simulated key press takes a few hundred nanoseconds instead of 10 ms.
4. **Wait forever** at the end once every filter's image has been written.

6.2 Create the ModelSim Command File

Complete sim/sim.do. It should compile the VHDL files in bottom-up order, start the testbench, add the important signals to the waveform viewer, and run the simulation. d8m.vhd and hdmi.vhd are not compiled, because the testbench does not use them.

```
vlib work

vcom -2008 ../vhdl/pixel_pkg.vhd
vcom -2008 ../vhdl/bram.vhd
vcom -2008 ../vhdl/digit_display.vhd
```

```
vcom -2008 ../vhdl/video_store.vhd
vcom -2008 ../vhdl/filter_pass.vhd
(... the rest of your filter files)
vcom -2008 ../vhdl/filters.vhd
vcom -2008 ../vhdl/image_processor.vhd
vcom -2008 ../vhdl/image_processor_tb.vhd

vsim -voptargs=+acc image_processor_tb

add wave -group "TESTBENCH"       sim:/image_processor_tb/*
add wave -group "IMAGE PROCESSOR" sim:/image_processor_tb/dut/*
add wave -group "VIDEO STORE"     sim:/image_processor_tb/dut/u_video_store/*
add wave -group "FILTERS"         sim:/image_processor_tb/dut/u_filters/*

run -all
```

6.3 Run and Verify

Open ModelSim, change to the sim directory, and run:

```
do sim.do
```

Each filter takes roughly 1.5 million clock cycles to stream in and read back, so the full run takes a while. Open the output images and verify that each filter produces the effects described above.



7. Synthesis

Once the design simulates correctly, implement it on the DE25-Standard.

7.1 Open the Quartus Prime Project

Open the provided project syn/image_processor.qpf in Quartus Prime. It is already set up with:

- Top-level entity: **image_processor_system**
- Device: Agilex 5, **A5ED013BB32AE4SCS**
- The PLL IP, VIDEO_PLL.ip
- The camera configuration Verilog files in vhdl/d8m/
- The timing constraints file, image_processor.sdc
- Add all the VHDL files from the vhdl directory to the project, except the testbench.

7.2 Assign FPGA Pins

Open **Assignments → Pin Planner** and assign the following signals. The I/O standards are already set in the project.

Port	FPGA Pin	I/O Standard	Description
clk_50	BM71	1.2-V	50 MHz board clock
reset_n	BM78	1.2-V	CPU reset button
keys_n[0]	BW59	1.2-V	KEY0: next filter
keys_n[1]	CA59	1.2-V	KEY1: previous filter
hex0[0]	BP81	1.2-V	HEX0 segment a
hex0[1]	BM81	1.2-V	HEX0 segment b
hex0[2]	BM89	1.2-V	HEX0 segment c
hex0[3]	BK89	1.2-V	HEX0 segment d
hex0[4]	BH92	1.2-V	HEX0 segment e
hex0[5]	BM92	1.2-V	HEX0 segment f
hex0[6]	BP92	1.2-V	HEX0 segment g
hex1[0]	CA78	1.2-V	HEX1 segment a
hex1[1]	BW78	1.2-V	HEX1 segment b
hex1[2]	BU78	1.2-V	HEX1 segment c
hex1[3]	BR78	1.2-V	HEX1 segment d
hex1[4]	BU81	1.2-V	HEX1 segment e
hex1[5]	BR81	1.2-V	HEX1 segment f
hex1[6]	CA89	1.2-V	HEX1 segment g
mipi_pixel_clk	BE43	3.3-V LVCMOS	D8M camera pixel clock
mipi_pixel_d[0]	BF40	3.3-V LVCMOS	D8M pixel data bit 0
mipi_pixel_d[1]	BK28	3.3-V LVCMOS	D8M pixel data bit 1
mipi_pixel_d[2]	BM31	3.3-V LVCMOS	D8M pixel data bit 2
mipi_pixel_d[3]	BM28	3.3-V LVCMOS	D8M pixel data bit 3
mipi_pixel_d[4]	BP31	3.3-V LVCMOS	D8M pixel data bit 4

Port	FPGA Pin	I/O Standard	Description
mipi_pixel_d[5]	BR31	3.3-V LVCMOS	D8M pixel data bit 5
mipi_pixel_d[6]	BU28	3.3-V LVCMOS	D8M pixel data bit 6
mipi_pixel_d[7]	BU31	3.3-V LVCMOS	D8M pixel data bit 7
mipi_pixel_d[8]	BW28	3.3-V LVCMOS	D8M pixel data bit 8
mipi_pixel_d[9]	BR22	3.3-V LVCMOS	D8M pixel data bit 9
mipi_pixel_hs	BR19	3.3-V LVCMOS	D8M line sync
mipi_pixel_vs	BM22	3.3-V LVCMOS	D8M frame sync
mipi_refclk	BF36	3.3-V LVCMOS	20 MHz reference clock to the MIPI bridge
mipi_cs_n	BM19	3.3-V LVCMOS	MIPI bridge chip select (active low)
mipi_reset_n	BK19	3.3-V LVCMOS	MIPI bridge reset (active low)
mipi_i2c_scl	CK2	3.3-V LVCMOS	MIPI bridge I2C clock
mipi_i2c_sda	CJ2	3.3-V LVCMOS	MIPI bridge I2C data
camera_i2c_scl	BF25	3.3-V LVCMOS	Camera sensor I2C clock
camera_i2c_sda	CF9	3.3-V LVCMOS	Camera sensor I2C data
camera_pwdn_n	BH19	3.3-V LVCMOS	Camera sensor power-down (active low)
hdmi_tx_clk	CH59	1.2-V	HDMI transmitter pixel clock
hdmi_tx_hs	BR109	3.3-V LVCMOS	HDMI horizontal sync
hdmi_tx_vs	BE107	3.3-V LVCMOS	HDMI vertical sync
hdmi_tx_de	BK109	3.3-V LVCMOS	HDMI data enable
hdmi_tx_d[0]	CD134	3.3-V LVCMOS	HDMI pixel data bit 0
hdmi_tx_d[1]	CD135	3.3-V LVCMOS	HDMI pixel data bit 1
hdmi_tx_d[2]	CG134	3.3-V LVCMOS	HDMI pixel data bit 2
hdmi_tx_d[3]	CG135	3.3-V LVCMOS	HDMI pixel data bit 3
hdmi_tx_d[4]	CH132	3.3-V LVCMOS	HDMI pixel data bit 4
hdmi_tx_d[5]	CF132	3.3-V LVCMOS	HDMI pixel data bit 5
hdmi_tx_d[6]	CF128	3.3-V LVCMOS	HDMI pixel data bit 6
hdmi_tx_d[7]	CK134	3.3-V LVCMOS	HDMI pixel data bit 7
hdmi_tx_d[8]	CL125	3.3-V LVCMOS	HDMI pixel data bit 8
hdmi_tx_d[9]	CF121	3.3-V LVCMOS	HDMI pixel data bit 9
hdmi_tx_d[10]	CF118	3.3-V LVCMOS	HDMI pixel data bit 10
hdmi_tx_d[11]	BU118	3.3-V LVCMOS	HDMI pixel data bit 11
hdmi_tx_d[12]	BR118	3.3-V LVCMOS	HDMI pixel data bit 12
hdmi_tx_d[13]	CA118	3.3-V LVCMOS	HDMI pixel data bit 13
hdmi_tx_d[14]	BW118	3.3-V LVCMOS	HDMI pixel data bit 14
hdmi_tx_d[15]	CL128	3.3-V LVCMOS	HDMI pixel data bit 15
hdmi_tx_d[16]	CL130	3.3-V LVCMOS	HDMI pixel data bit 16
hdmi_tx_d[17]	CK125	3.3-V LVCMOS	HDMI pixel data bit 17

Port	FPGA Pin	I/O Standard	Description
hdmi_tx_d[18]	CK128	3.3-V LVCMOS	HDMI pixel data bit 18
hdmi_tx_d[19]	BF111	3.3-V LVCMOS	HDMI pixel data bit 19
hdmi_tx_d[20]	BH109	3.3-V LVCMOS	HDMI pixel data bit 20
hdmi_tx_d[21]	BE115	3.3-V LVCMOS	HDMI pixel data bit 21
hdmi_tx_d[22]	BF115	3.3-V LVCMOS	HDMI pixel data bit 22
hdmi_tx_d[23]	BU109	3.3-V LVCMOS	HDMI pixel data bit 23
hdmi_tx_int	BE111	3.3-V LVCMOS	HDMI transmitter interrupt (active low)
hdmi_i2c_scl	BF120	3.3-V LVCMOS	HDMI transmitter I2C clock
hdmi_i2c_sda	BH118	3.3-V LVCMOS	HDMI transmitter I2C data

7.4 Compile and Synthesize

Run the complete Quartus Prime compilation flow. The compilation should complete without errors. Review the compilation reports and verify that:

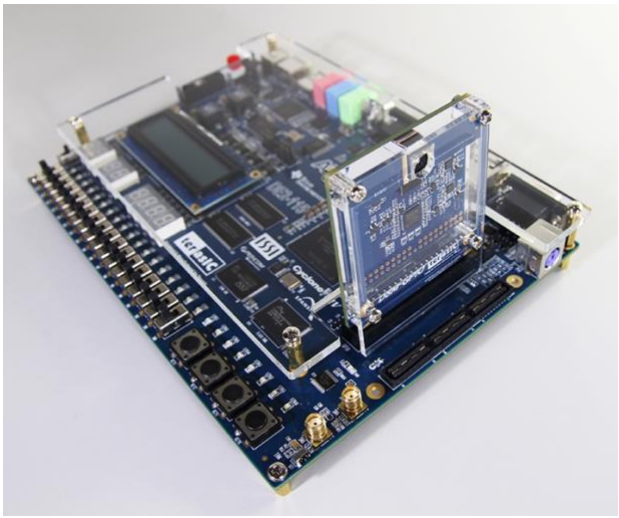
- **image_processor_system** is the top-level entity, and all top-level ports have pin assignments.
- **Memory:** the Fitter's resource usage summary shows about **300 M20K blocks** used for the frame buffer. If the frame buffer was built from logic instead, the logic usage will be far too high to fit, which usually means the memory has a reset or its read address is not registered.
- **Timing:** the Timing Analyzer shows the 100 MHz system clock, the 20 MHz reference clock and the 25 MHz camera clock, with no setup or hold violations.
- **Constraints applied:** the set_max_skew and set_net_delay constraints matched registers. If Quartus warns that get_registers matched nothing, check that the video store instance is labelled u_video_store.

8. Hardware Setup and Testing

8.1 Connect the Hardware

Make all connections with the FPGA board powered off.

- Connect the **D8M camera** to the DE25-Standard's GPIO expansion header. Check the D8M manual for the correct header and orientation before applying power.
- Connect a monitor to the board's **HDMI** port.
- Connect the board to the computer through its **USB-Blaster** connection.



8.2 Program and Test the System

Power on the board and the monitor, open **Tools → Programmer** in Quartus Prime, select the .sof file, and program the FPGA. The camera takes about a second to configure, after which the monitor should show the live camera image with filter 0 (passthrough), and HEX1 and HEX0 should show **00**.

Step up/down through all eight filters with KEY0 or KEY1, and verify that the HEX display shows each filter number and that the picture matches the simulation results. Move objects in front of the camera, and verify that the edge and cartoonify filters follow their outlines in real time.

9. Submission

Zip your folder, name it **CE355_Lab5_Team_[N].zip**, and submit it to Canvas. Make sure to clean out any synthesis and simulation work directories, and do not include the generated output_*.bmp files.

9.1 Report

Submit a report of **no more than 2–3 pages**, figures included. Be concise: every figure must be referenced and explained in the text, and must be legible — crop and zoom rather than pasting a full-desktop screenshot. Cover the following.

- **Simulation.** The transcript output from the simulator log window, including one or two annotated waveforms and explanation that shows how you know the design is working. Include explanation of any test cases you ran. Include any output rendered in simulation (such as images or text).
- **Schematic.** An RTL Viewer image showing your structure and correct connections, including instantiation of key components — block RAMs, shift registers, FIFOs, PLLs, DSP blocks, IP. Note anything the tool inferred that you did not intend: a latch, LUT-based memory instead of a block RAM, a multiplier built from logic.

- **Resources.** Based on the synthesis results, determine if the resources used align with what is expected, for example: number of multipliers (DSPs), memory bits and the blocks, approximate register count, shift registers, and any other IPs. Explain any significant differences.
- **Timing.** fMAX and worst-case setup slack for each clock, the registers the critical path runs between, and why it is the slowest path. “Timing met” is not an analysis. Explain any changes you made to your design to make timing if it wasn’t met initially.
- **Optimizations.** Discuss any optimizations or techniques you implemented and its significance.
- **Hardware and attribution.** Which hardware tests passed, and note any differences between simulation and hardware.
- **Screenshots** of the hardware working correctly both simulation and on the board.

9.2 Demonstration

You will also need to demo the hardware setup and simulation during TA office hours. Be prepared to explain how the system works, including why the design needs a frame buffer, how the two clock domains are kept apart, how the read address and valid signal stay aligned with the frame buffer’s latency, how each filter works, why sepia needs saturation, why the edge filter only updates its memory on valid pixels, and how the button debouncing works.